JAVASCRIPT

COMPLETE MASTER REFERENCE

Core language • Built-in objects • String & Array methods • Functions • DOM • Async JavaScript • Projects

This expanded edition fixes the scope: the earlier PDF focused mainly on functions. This guide covers the wider JavaScript language, its commonly used built-in methods, syntax, browser APIs and practical patterns.

How to use it: read a concept, run its code, change the values, and predict the output before executing it.

Contents and scope

• 1. JavaScript foundations and execution

• 2. Variables, data types and type conversion

• 3. Operators and control flow

• 4. Loops and iteration

• 5. Functions: forms, parameters, methods and concepts

• 6. String methods and examples

• 7. Array methods and examples

• 8. Objects and object methods

• 9. Number, Math, Date and JSON

• 10. Map, Set, WeakMap and WeakSet

• 11. Destructuring, spread, rest and optional chaining

• 12. Errors and debugging

• 13. Asynchronous JavaScript: callbacks, promises, async/await

• 14. DOM and browser events

• 15. Modules, classes, prototypes and OOP

• 16. Regular expressions

• 17. Iterators, generators and symbols

• 18. Storage, fetch and browser APIs

• 19. Common interview comparisons and pitfalls

• 20. Practical programs and exercises

Note: JavaScript is a large language and browser APIs keep evolving. This reference aims to be broad and practical, not a claim that every browser API or every rarely used legacy method is listed.

1. JavaScript foundations

JavaScript is a high-level, dynamically typed programming language. It is used in browsers and also outside browsers through runtimes such as Node.js. The ECMAScript specification defines the core language; browser APIs such as the DOM are provided by the browser.

First program

```
console.log("Hello, JavaScript!");

let name = "Aman";
console.log("Hello " + name);
```

console.log() prints a value to the developer console. It is a debugging tool, not a language output statement like Python's print().

Comments

```
// Single-line comment

/*
  Multi-line
  comment
*/
```

Statements, expressions and blocks

An expression produces a value, such as 2 + 3. A statement performs an action, such as declaring a variable or an if statement. Curly braces create blocks in many constructs.

2. Variables and data types

Keyword	Reassign?	Redeclare in same scope?	Scope
var	Yes	Yes	Function/global (legacy)
let	Yes	No	Block

Keyword	Reassign?	Redeclare in same scope?	Scope
const	No reassignment	No	Block

```
let score = 10;
score = 20;

const country = "India";
// country = "Japan"; // TypeError

var oldStyle = "avoid var in modern code";
```

const prevents rebinding the variable; it does not make an object or array deeply immutable.

Primitive data types

Type	Example
string	'hello'
number	10, 3.14, NaN, Infinity
bigint	1234567890123456789 0n
boolean	true / false
undefined	let x;
null	let x = null;
symbol	Symbol('id')

Reference types

Objects, arrays and functions are objects/reference values. Variables hold references to them.

```
const user = { name: "Riya" };
const sameUser = user;
sameUser.name = "Aman";
console.log(user.name); // Aman
```

typeof and type checking

```
console.log(typeof "hi");        // "string"
console.log(typeof 12);            // "number"
console.log(typeof true);          // "boolean"
console.log(typeof undefined);     // "undefined"
console.log(typeof null);          // "object" (historic quirk)
console.log(Array.isArray([]));    // true
console.log(Number.isNaN(NaN));   // true
```

Use `Array.isArray()` to check arrays. Use `Number.isNaN()` for a reliable NaN check.

3. Operators

Category	Operators	Purpose
Arithmetic	+ - * / % **	Math operations
Assignment	= += -= *= /= %=	Assign/update
Comparison	== === != !== > < >= <=	Compare values
Logical	&& || !	Boolean logic
Nullish	?? ??=	Fallback only for null/undefined
Optional chaining	?.	Safely access nested values
Conditional	condition ? a : b	Choose one expression
Bitwise	& | ^ ~ << >> >>>	Operate on 32-bit integer representations
Type	typeof, instanceof	Inspect type/relationship

Equality: == vs ===

```
console.log(5 == "5");  // true: coercion
console.log(5 === "5"); // false: different types
```

Prefer strict equality === unless you have a clear reason to allow coercion.

Logical short-circuiting

```
console.log(true && "yes"); // "yes"
console.log(false || "fallback"); // "fallback"
console.log(null ?? "default"); // "default"
console.log(0 ?? 100); // 0
```

Spread and exponentiation examples

```
console.log(2 ** 3); // 8

const a = [1, 2];
const b = [...a, 3, 4];
console.log(b); // [1, 2, 3, 4]
```

4. Conditional statements and loops

```
const marks = 78;
if (marks >= 90) {
  console.log("A");
} else if (marks >= 60) {
  console.log("B");
} else {
  console.log("Needs improvement");
}

const day = 2;
switch (day) {
  case 1: console.log("Monday"); break;
  case 2: console.log("Tuesday"); break;
  default: console.log("Other day");
}
```

Loops

```
for (let i = 1; i <= 3; i++) console.log(i);

let n = 1;
while (n <= 3) {
  console.log(n);
  n++;
}

let x = 1;
do {
  console.log(x);
  x++;
} while (x <= 3);

const colorsList = ["red", "blue"];
for (const color of colorsList) console.log(color);

const personObj = { name: "Aman", age: 20 };
for (const key in personObj) console.log(key, personObj[key]);
```

for...of iterates iterable values (arrays, strings, maps). **for...in** iterates enumerable property keys; for arrays, prefer for...of or array methods.

5. Functions: complete practical section

Declaration, expression and arrow

```
function add(a, b) { return a + b; }

const multiply = function (a, b) {
  return a * b;
};

const square = n => n * n;
console.log(add(2, 3), multiply(2, 3), square(4));
```

Parameters, defaults, rest and return

```
function greet(name = "Guest") {
  return `Hello, ${name}`;
}

function sum(...values) {
  return values.reduce((total, value) => total + value, 0);
}
```

```
console.log(greet());
console.log(sum(2, 4, 6));
```

Function object methods

Method	Use	Key point
call(thisArg, a, b)	Call function now	Arguments passed separately
apply(thisArg, [a,b])	Call function now	Arguments passed as array-like
bind(thisArg, a)	Create a bound function	Does not call immediately
toString()	Get source-like text	Formatting is implementation-dependent
[Symbol.hasInstance]	Customize instanceof	Advanced feature

```
function describe(city, country) {
  return `${this.name} lives in ${city}, ${country}`;
}
const person = { name: "Mira" };

console.log(describe.call(person, "Pune", "India"));
console.log(describe.apply(person, ["Pune", "India"]));

const later = describe.bind(person, "Pune");
console.log(later("India"));
```

Function properties

Property	Meaning
name	Function name
length	Number of formal parameters before first default/rest parameter
prototype	Prototype used by constructible functions
arguments / caller	Legacy/deprecated or restricted; avoid

Callback, higher-order function and closure

```
function runTask(callback) {
  console.log("Starting");
  callback();
}
runTask(() => console.log("Finished"));

function makeCounter() {
  let count = 0;
  return () => ++count;
}
const counter = makeCounter();
console.log(counter(), counter()); // 1 2
```

A callback is passed into another function. A higher-order function accepts or returns functions. A closure retains access to variables from its lexical scope.

Recursion and IIFE

```
function factorial(n) {
  if (n <= 1) return 1;
  return n * factorial(n - 1);
}
console.log(factorial(5)); // 120

(() => console.log("Runs immediately"))();
```

6. String methods — important complete set

Strings are immutable: methods return a new string; they do not change the original string.

Method	Purpose	Example result
length (property)	Number of UTF-16 code units	'hello'.length → 5
at(i)	Character at index; supports negative index	'hello'.at(-1) → 'o'
charAt(i)	Character at index	'cat'.charAt(1) → 'a'

Method	Purpose	Example result
charCodeAt(i)	UTF-16 code unit	'A'.charCodeAt(0) → 65
codePointAt(i)	Unicode code point	'A'.codePointAt(0) → 65
concat()	Join strings	'a'.concat('b') → 'ab'
includes()	Contains substring?	'hello'.includes('ell') → true
startsWith()	Starts with substring?	'hello'.startsWith('he') → true
endsWith()	Ends with substring?	'hello'.endsWith('lo') → true
indexOf()	First match index; -1 if absent	'banana'.indexOf('na') → 2
lastIndexOf()	Last match index	'banana'.lastIndexOf('na') → 4
slice(start,end)	Extract portion; end excluded	'JavaScript'.slice(0,4) → 'Java'
substring(start,end)	Extract; negative treated as 0	'JavaScript'.substring(4,0) → 'Java'
toUpperCase()	Uppercase	'hi'.toUpperCase() → 'HI'
toLowerCase()	Lowercase	'HI'.toLowerCase() → 'hi'
trim()	Remove both-side whitespace	' hi '.trim() → 'hi'
trimStart()/trimLeft()	Remove start whitespace	' hi'.trimStart() → 'hi'
trimEnd()/trimRight()	Remove end whitespace	'hi '.trimEnd() → 'hi'
padStart()	Pad at beginning	'7'.padStart(3,'0') → '007'
padEnd()	Pad at end	'7'.padEnd(3,'0') → '700'
repeat(n)	Repeat string	'ha'.repeat(3) → 'hahaha'
replace()	Replace first match (string search)	'a-a'.replace('-','+') → 'a+a'
replaceAll()	Replace all matches	'a-a'.replaceAll('-','+') → 'a+a'
split()	Split into array	'a,b'.split(',') → ['a','b']
match()	Matches using regex	'abc'.match(/[a-z]/g)
matchAll()	Iterator of all regex matches	'a1b2'.matchAll(/\d/g)
search()	Regex match index	'abc'.search(/b/) → 1
localeCompare()	Locale-aware comparison	'a'.localeCompare('b') → negative
normalize()	Unicode normalization	text.normalize('NFC')

String methods — code and explanation

```
const text = "  JavaScript is powerful  ";

console.log(text.length);             // length includes spaces
console.log(text.trim());             // remove outside spaces
console.log(text.toUpperCase());      // uppercase copy
console.log(text.includes("Script")); // true
console.log(text.slice(2, 12));       // "JavaScript"
console.log(text.replace("powerful", "great"));
console.log("red,green,blue".split(",")); // array
console.log("5".padStart(3, "0"));     // "005"
console.log("ha".repeat(3));          // "hahaha"

// Original string is unchanged:
console.log(text);
```

slice vs substring: slice supports negative indexes and returns empty when end is before start; substring swaps start/end if needed and treats negative values as zero.

replace vs replaceAll: replace with a string search replaces only the first occurrence; replaceAll replaces all literal occurrences. A global regular expression can also replace all matches.

Template literals

```
const name = "Aman";
const score = 95;
console.log(`${name} scored ${score}%`);

const message = `This is
```

```
a multi-line
string.`;
```

7. Array methods — essential full reference

Arrays are ordered, zero-indexed collections. Most array methods are grouped by whether they mutate the original array or return a new value.

Mutating methods

Method	Purpose	Example / note
push()	Add at end	arr.push(9)
pop()	Remove from end; returns removed item	arr.pop()
unshift()	Add at beginning	arr.unshift(0)
shift()	Remove first; returns removed item	arr.shift()
splice()	Insert/remove/replace in place	arr.splice(1,2,'x')
sort()	Sort in place	arr.sort((a,b)=>a-b) for numbers
reverse()	Reverse in place	arr.reverse()
fill()	Fill range with value	arr.fill(0,1,3)
copyWithin()	Copy part within same array	arr.copyWithin(0,2)

Non-mutating / value-returning methods

Method	Purpose	Example / note
at()	Get element by index; negative supported	arr.at(-1)
concat()	Join arrays/values	a.concat(b)
join()	Convert elements to string	arr.join('-')
slice()	Copy a portion; end excluded	arr.slice(1,3)
includes()	Check value exists	arr.includes(5)
indexOf()	First matching index	arr.indexOf(5)
lastIndexOf()	Last matching index	arr.lastIndexOf(5)
toString()	String representation	arr.toString()
toLocaleString()	Locale-aware string conversion	arr.toLocaleString()
toReversed()	Non-mutating reverse	arr.toReversed()
toSorted()	Non-mutating sort	arr.toSorted((a,b)=>a-b)
toSpliced()	Non-mutating splice equivalent	arr.toSpliced(1,1)
with()	Copy with one index replaced	arr.with(1,99)

Iteration and transformation methods

Method	Purpose	Returns
forEach()	Run callback for each element	undefined
map()	Transform every element	New array
filter()	Keep elements passing condition	New array
reduce()	Accumulate left to right	Single accumulated result
reduceRight()	Accumulate right to left	Single accumulated result
find()	First matching element	Element / undefined
findIndex()	Index of first match	Index / -1
findLast()	Last matching element	Element / undefined
findLastIndex()	Index of last match	Index / -1

Method	Purpose	Returns
some()	At least one passes?	Boolean
every()	All pass?	Boolean
flat()	Flatten nested arrays	New array
flatMap()	Map then flatten one level	New array
entries()	Iterator of [index,value]	Iterator
keys()	Iterator of indexes	Iterator
values()	Iterator of values	Iterator

Array static methods

Method	Purpose
Array.isArray(value)	Check whether value is an array
Array.from(iterable, mapFn?)	Create array from iterable/array-like
Array.of(...items)	Create array from arguments

Array methods — detailed code

```
const nums = [1, 2, 3, 4, 5];

console.log(nums.map(n => n * 2));       // [2,4,6,8,10]
console.log(nums.filter(n => n % 2 === 0)); // [2,4]
console.log(nums.find(n => n > 3));      // 4
console.log(nums.findIndex(n => n > 3)); // 3
console.log(nums.some(n => n > 4));      // true
console.log(nums.every(n => n > 0));     // true
console.log(nums.reduce((sum, n) => sum + n, 0)); // 15

// Mutating examples:
const items = [3, 1, 2];
items.push(4);              // [3,1,2,4]
items.sort((a, b) => a-b);  // [1,2,3,4]
items.splice(1, 1, 99);     // [1,99,3,4]

// Non-mutating examples:
const original = [10, 20, 30];
const copy = original.slice(1); // [20,30]
const changed = original.with(1, 99); // [10,99,30]
console.log(original); // unchanged
```

sort() — important warning

```
const values = [10, 2, 30, 4];
values.sort();
console.log(values); // [10, 2, 30, 4] lexicographic sort!

values.sort((a, b) => a - b);
console.log(values); // [2, 4, 10, 30] numeric ascending
```

Default sort compares string representations. Supply a comparator for numeric sorting.

splice vs slice

slice()	splice()
Does not mutate original	Mutates original
Extracts a shallow copy	Removes/inserts/replaces elements
slice(start,end)	splice(start, deleteCount, ...items)

Chaining

```
const result = [1, 2, 3, 4, 5, 6]
  .filter(n => n % 2 === 0)
  .map(n => n * n)
  .reduce((sum, n) => sum + n, 0);

console.log(result); // 56
```

8. Objects and object methods

An object stores key-value pairs. Properties can hold data or functions (methods).

```
const student = {
  name: "Riya",
  marks: 92,
  greet() {
    return `Hi, I am ${this.name}`;
  }
};
console.log(student.name);
console.log(student["marks"]);
console.log(student.greet());
```

Object static method	Purpose
Object.keys(obj)	Array of own enumerable string keys
Object.values(obj)	Array of own enumerable values
Object.entries(obj)	Array of [key,value] pairs
Object.fromEntries(entries)	Create object from key-value pairs
Object.assign(target, ...sources)	Copy enumerable own properties
Object.create(proto)	Create object with chosen prototype
Object.hasOwn(obj, key)	Check own property
Object.freeze(obj)	Prevent changes to own properties (shallow)
Object.seal(obj)	Prevent add/remove; existing values may change if writable
Object.is(a,b)	SameValue comparison
Object.getOwnPropertyNames(obj)	All own string property names
Object.getOwnPropertySymbols(obj)	Own symbol keys
Object.getPrototypeOf(obj)	Get prototype
Object.setPrototypeOf(obj, proto)	Set prototype; generally avoid for performance

```
const profile = { name: "Aman", age: 20 };
console.log(Object.keys(profile));   // ["name","age"]
console.log(Object.values(profile)); // ["Aman",20]
console.log(Object.entries(profile));

const entries = [["city", "Delhi"], ["active", true]];
console.log(Object.fromEntries(entries));

console.log(Object.hasOwn(profile, "name")); // true

const copy = { ...profile }; // shallow copy
```

Shallow copy: spread syntax and Object.assign copy only the top-level properties. Nested objects remain shared references.

Property descriptors

```
const obj = {};
Object.defineProperty(obj, "id", {
  value: 101,
  writable: false,
  enumerable: true,
  configurable: false
});
console.log(obj.id); // 101
```

9. Number, Math, Date and JSON

Number methods and properties

API	Purpose
Number()	Convert to number
Number.isNaN()	Strict NaN test
Number.isFinite()	Finite number test

API	Purpose
Number.isInteger()	Integer test
Number.isSafeInteger()	Safe integer test
Number.parseInt()	Parse integer
Number.parseFloat()	Parse decimal
toFixed(digits)	Format fixed decimal places as string
toPrecision(digits)	Format to significant digits
toString(radix)	Convert number to string in base
Number.MAX_VALUE / MIN_VALUE	Numeric limits
Number.MAX_SAFE_INTEGER	Largest safe integer

```
console.log(Number("42"));          // 42
console.log(Number("hello"));        // NaN
console.log(Number.parseInt("25px")); // 25
console.log(Number.parseFloat("3.14kg")); // 3.14
console.log((12.345).toFixed(2));    // "12.35"
console.log(Number.isInteger(4.0));  // true
```

Math object

Method/property	Purpose
Math.abs(x)	Absolute value
Math.ceil(x)	Round up
Math.floor(x)	Round down
Math.round(x)	Nearest integer
Math.trunc(x)	Remove decimal part
Math.max(...values)	Largest
Math.min(...values)	Smallest
Math.pow(a,b)	Power
Math.sqrt(x)	Square root
Math.cbrt(x)	Cube root
Math.random()	Pseudo-random number in [0,1)
Math.sign(x)	Sign
Math.hypot(...values)	Square root of sum of squares
Math.log(x), log10(), log2()	Logarithms
Math.sin(), cos(), tan()	Trigonometry (radians)
Math.PI, Math.E	Constants

```
console.log(Math.floor(4.9)); // 4
console.log(Math.ceil(4.1));  // 5
console.log(Math.round(4.5)); // 5
console.log(Math.max(2, 8, 5)); // 8
console.log(Math.sqrt(81)); // 9

// Random integer from 1 to 10 inclusive
const random = Math.floor(Math.random() * 10) + 1;
```

Date

```
const now = new Date();
console.log(now);
console.log(now.getFullYear());
console.log(now.getMonth()); // 0 = January
console.log(now.getDate());  // day of month
console.log(now.getDay());   // 0 = Sunday
console.log(now.getHours());
console.log(now.toISOString());
console.log(Date.now());     // milliseconds since epoch
```

```
const birthday = new Date("2005-05-20");
```

Month indexes start at 0. Date parsing of non-standard strings can vary; ISO-style date strings are safer.

JSON

```
const user = { name: "Aman", age: 20 };
const jsonText = JSON.stringify(user);
console.log(jsonText); // JSON string

const parsed = JSON.parse(jsonText);
console.log(parsed.name); // Aman
```

JSON.stringify converts supported JavaScript data to JSON text. JSON.parse converts valid JSON text back into values. JSON is a data format, not a JavaScript object method.

10. Map, Set, WeakMap and WeakSet

Map

```
const map = new Map();
map.set("name", "Riya");
map.set(1, "one");
console.log(map.get("name"));
console.log(map.has(1));
console.log(map.size);
map.delete(1);
map.forEach((value, key) => console.log(key, value));
map.clear();
```

Map stores key-value pairs and allows keys of any type. Common methods: set(), get(), has(), delete(), clear(); size is a property.

Set

```
const unique = new Set([1, 2, 2, 3]);
unique.add(4);
console.log(unique.has(2)); // true
console.log(unique.size);   // 4
unique.delete(1);
console.log([...unique]);   // [2,3,4]
unique.clear();
```

Set stores unique values. Common methods: add(), has(), delete(), clear(); size is a property.

WeakMap and WeakSet

WeakMap keys must be objects (or non-registered symbols in modern specifications); WeakSet stores objects (and supported non-registered symbols). They are weakly held and are not enumerable. They do not expose size or clear(). Use them for metadata tied to object lifetimes.

11. Destructuring, spread, rest and optional chaining

```
const user = { name: "Aman", age: 20 };
const { name, age } = user;

const nums = [10, 20, 30];
const [first, second] = nums;

const more = [...nums, 40]; // spread
function total(...values) { // rest
  return values.reduce((a,b) => a+b, 0);
}

const city = user.address?.city ?? "Unknown";
console.log(name, first, more, total(1,2,3), city);
```

Spread expands an iterable/object's properties. Rest collects remaining values. The syntax looks similar but its job depends on where it appears.

12. Errors and debugging

```
try {
  const data = JSON.parse("{bad json}");
  console.log(data);
} catch (error) {
  console.error("Something went wrong:", error.message);
} finally {
  console.log("This always runs");
}
```

```
function withdraw(balance, amount) {
  if (amount > balance) {
    throw new Error("Insufficient balance");
  }
  return balance - amount;
}
```

Common built-in errors include Error, TypeError, ReferenceError, SyntaxError, RangeError and URIError. Use throw to raise an error and try/catch to handle it.

13. Asynchronous JavaScript

Callback

```
setTimeout(() => {
  console.log("Runs after about 1 second");
}, 1000);
```

Promise

```
const task = new Promise((resolve, reject) => {
  const success = true;
  if (success) resolve("Completed");
  else reject(new Error("Failed"));
});

task
  .then(value => console.log(value))
  .catch(error => console.error(error))
  .finally(() => console.log("Finished"));
```

Promise static methods

Method	Purpose
Promise.resolve(value)	Create fulfilled promise
Promise.reject(reason)	Create rejected promise
Promise.all(iterable)	Fulfill when all fulfill; rejects on first rejection
Promise.allSettled(iterable)	Wait for every promise to settle
Promise.race(iterable)	Settle with first settled promise
Promise.any(iterable)	Fulfill with first fulfilled promise; rejects if all reject

async / await

```
async function getTodo() {
  try {
    const response = await fetch("https://jsonplaceholder.typicode.com/todos/1");
    if (!response.ok) throw new Error("HTTP error " + response.status);
    const todo = await response.json();
    console.log(todo);
  } catch (error) {
    console.error(error);
  }
}
getTodo();
```

An async function always returns a Promise. await pauses the async function, not the entire JavaScript runtime.

14. DOM and browser events

The DOM (Document Object Model) represents an HTML document as a tree of objects. It is a browser API, not part of the core ECMAScript language.

```
// Select elements
const heading = document.querySelector("#title");
const buttons = document.querySelectorAll(".btn");

// Read and update content
heading.textContent = "Welcome";
heading.classList.add("active");

// Create and insert element
const li = document.createElement("li");
li.textContent = "New item";
document.querySelector("ul").append(li);
```

```
// Event listener
document.querySelector("#save").addEventListener("click", event => {
  console.log("Button clicked", event.target);
});
```

Common DOM selectors and methods

API	Purpose
getElementById()	Select by id
getElementsByClassName()	Live collection by class
getElementsByTagName()	Live collection by tag
querySelector()	First CSS selector match
querySelectorAll()	Static NodeList of matches
createElement()	Create element
append()/prepend()	Insert nodes/content
remove()	Remove element
replaceWith()	Replace element
setAttribute()/getAttribute()	Manage attributes
classList.add/remove/toggle/contains()	Manage CSS classes
addEventListener()	Listen for events
removeEventListener()	Remove matching listener

Events and form values

```
const form = document.querySelector("form");
form.addEventListener("submit", event => {
  event.preventDefault(); // stop page reload
  const formData = new FormData(form);
  console.log(formData.get("email"));
});
```

Common events: click, input, change, submit, keydown, keyup, mouseover, mouseout, load, DOMContentLoaded. Use event.preventDefault() to prevent the browser's default action.

15. Classes, prototypes and OOP

```
class Person {
  constructor(name) {
    this.name = name;
  }
  greet() {
    return `Hello ${this.name}`;
  }
}

class Student extends Person {
  constructor(name, course) {
    super(name);
    this.course = course;
  }
  info() {
    return `${this.name}: ${this.course}`;
  }
}

const s = new Student("Aman", "CSE");
console.log(s.greet());
console.log(s.info());
console.log(s instanceof Person); // true
```

constructor initializes instances. extends creates inheritance. super() calls the parent constructor. Methods are placed on the class prototype. JavaScript classes use prototype-based inheritance underneath.

Prototype essentials

```
const base = { role: "user" };
const account = Object.create(base);
account.name = "Mira";
```

```
console.log(account.role); // inherited property
console.log(Object.getPrototypeOf(account) === base); // true
```

16. Modules

```
// math.js
export function add(a, b) {
  return a + b;
}
export const PI_VALUE = 3.14159;

// app.js
import { add, PI_VALUE } from "./math.js";
console.log(add(2, 3), PI_VALUE);
```

Named exports/imports use braces. A module can also have one default export. In browsers, load module scripts with `type="module"`. Node.js also supports ECMAScript modules.

17. Regular expressions

```
const text = "Order number: 4829";
const pattern = /\d+/;

console.log(pattern.test(text)); // true
console.log(text.match(/\d+/)); // match details
console.log(text.replace(/\d+/, "XXXX"));

const emailLike = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
console.log(emailLike.test("a@example.com"));
```

Regex token	Meaning
\d	Digit
\w	Word character
\s	Whitespace
.	Any character except line terminators by default
^ / $	Start / end
* / + / ?	Zero or more / one or more / optional
{m,n}	Repetition range
[abc] / [^abc]	Character set / negated set
( )	Capture group
(?:)	Non-capturing group
|	Alternation
g / i / m / s / u / y	Common flags

18. Iterators, generators and symbols

```
function* ids() {
  yield 1;
  yield 2;
  yield 3;
}
for (const id of ids()) {
  console.log(id);
}

const iterator = [10, 20][Symbol.iterator]();
console.log(iterator.next()); // { value:10, done:false }

const uniqueKey = Symbol("id");
const record = { [uniqueKey]: 123 };
console.log(record[uniqueKey]);
```

An iterable provides a Symbol.iterator method. An iterator returns objects with value and done. Generators make it easier to create iterators. Symbols are unique primitive values often used as non-string property keys.

19. Storage, fetch and useful browser APIs

localStorage and sessionStorage

```
localStorage.setItem("theme", "dark");
console.log(localStorage.getItem("theme"));
localStorage.removeItem("theme");
// localStorage.clear(); // removes all keys for this origin

sessionStorage.setItem("step", "2");
```

Storage APIs store strings. Convert objects with JSON.stringify/JSON.parse. localStorage persists across browser sessions; sessionStorage is scoped to the tab/session.

Fetch API

```
async function loadUsers() {
  const response = await fetch("/api/users");
  if (!response.ok) {
    throw new Error(`HTTP ${response.status}`);
  }
  return await response.json();
}
```

fetch() returns a Promise. HTTP error status codes do not automatically reject the Promise, so check response.ok.

Other browser APIs to know

API	Use
setTimeout()/clearTimeout()	Schedule/cancel one timer
setInterval()/clearInterval()	Repeat/cancel timer
URL / URLSearchParams	Parse and build URLs/query strings
Intl	Locale-aware number/date/string formatting
AbortController	Cancel fetch or other abortable work
navigator.clipboard	Clipboard access (permission/secure context rules apply)
Geolocation API	Location access with user permission
History API	Manage session history
IntersectionObserver	Observe element visibility
ResizeObserver	Observe element size
requestAnimationFrame()	Schedule visual updates before repaint

20. Common comparisons and mistakes

Comparison	Difference
map vs forEach	map returns transformed array; forEach returns undefined
filter vs find	filter returns all matches in an array; find returns first match
some vs every	some checks at least one; every checks all
slice vs splice	slice copies a range; splice mutates by removing/inserting
push vs unshift	push adds to end; unshift adds to beginning
pop vs shift	pop removes end; shift removes beginning
for...of vs for...in	values of iterable vs enumerable keys
== vs ===	coercive equality vs strict equality
null vs undefined	intentional absence vs not assigned/missing
call/apply vs bind	call/apply execute now; bind returns a new function
localStorage vs sessionStorage	persistent vs tab/session lifetime

• Use const by default; use let when reassignment is needed.

• Prefer === over ==.

• Do not mutate arrays accidentally; know which methods mutate.

• Remember sort() is lexicographic by default.

• Do not use async/await without error handling where failures matter.

• Do not assume fetch rejects on HTTP 404/500; check response.ok.

• Do not use innerHTML with untrusted input; prefer textContent.

• Remember object/array copies with spread are shallow.

• Use addEventListener instead of inline HTML event attributes.

• Do not confuse browser APIs (DOM, localStorage, fetch) with core language methods.

21. Practical programs

Reverse a string

```
function reverseString(str) {
  return str.split("").reverse().join("");
}
console.log(reverseString("hello")); // olleh
```

Palindrome check

```
function isPalindrome(str) {
  const cleaned = str.toLowerCase().replace(/[^a-z0-9]/g, "");
  return cleaned === cleaned.split("").reverse().join("");
}
console.log(isPalindrome("Madam")); // true
```

Count character frequency

```
function frequency(text) {
  const result = {};
  for (const char of text) {
    result[char] = (result[char] || 0) + 1;
  }
  return result;
}
console.log(frequency("banana"));
```

Remove duplicate array values

```
const values = [1, 2, 2, 3, 3, 4];
const uniqueValues = [...new Set(values)];
console.log(uniqueValues); // [1,2,3,4]
```

Find largest number

```
function largestNumber(numbers) {
  if (numbers.length === 0) return undefined;
  return Math.max(...numbers);
}
console.log(largestNumber([4, 9, 2, 15])); // 15
```

Group objects by property

```
const people = [
  { name: "A", team: "red" },
  { name: "B", team: "blue" },
  { name: "C", team: "red" }
];

const grouped = Object.groupBy(people, person => person.team);
console.log(grouped);
```

Object.groupBy is a modern JavaScript feature; check runtime support for older browsers/environments.

22. Practice set

■ Explain var, let and const with scope examples.

■ Write a program to count vowels in a string.

■ Demonstrate at least 12 String methods with output.

■ Demonstrate mutating and non-mutating Array methods.

■ Use map, filter and reduce to process student marks.

■ Sort numbers ascending and descending with a comparator.

■ Convert an object to entries and back using Object.entries/fromEntries.

■ Build a Map and Set and explain their differences.

■ Write a closure-based bank balance tracker.

■ Demonstrate call, apply and bind.

■ Create a Promise and consume it using async/await.

■ Fetch data and handle network and HTTP errors.

■ Create a DOM element and attach a click listener.

■ Build a class hierarchy using extends and super.

■ Write a regex to validate a basic email-like string.

■ Explain shallow copy vs deep copy with nested objects.

23. Recommended learning order

• Foundations → variables → types → operators → conditions → loops.

• Functions → scope → this → callbacks → closures → recursion.

• Strings → arrays → objects → built-in objects (Math, Date, JSON, Map, Set).

• Destructuring → spread/rest → errors → modules → classes/prototypes.

• Promises → async/await → fetch → DOM → events → storage.

• Build projects: calculator, to-do list, quiz app, weather app, expense tracker.

Final note: This is a broad JavaScript language and browser reference. For a truly exhaustive reference of every edge case and specification detail, use the ECMAScript and MDN documentation alongside this guide.